

Clean Code - Chapter 8: Boundaries (Complete Notes + Logging Wrapper)

1. Boundaries Concept

Boundaries are where our system interacts with external code such as libraries, APIs, or other systems. We must control how we interact with them.

2. Problem with Third-Party Code (Map)

```
Map sensors = new HashMap();
Sensor s = (Sensor) sensors.get(sensorId);
```

Improved with Generics

```
Map<String, Sensor> sensors = new HashMap<>();
Sensor s = sensors.get(sensorId);
```

Best Practice (Encapsulation)

```
public class Sensors {
    private Map<String, Sensor> sensors = new HashMap<>();

    public Sensor getById(String id) {
        return sensors.get(id);
    }
}
```

3. Learning Tests (Log Example)

```
@Test
public void testLogAddAppender() {
    Logger logger = Logger.getLogger("MyLogger");
    logger.removeAllAppenders();
    logger.addAppender(new ConsoleAppender(
        new PatternLayout("%p %t %m%n"),
        ConsoleAppender.SYSTEM_OUT));
    logger.info("hello");
}
```

4. Logging Library Wrapper (IMPORTANT)

Bad Example (Direct Usage)

```
import org.apache.log4j.Logger;

public class UserService {
    private static final Logger logger = Logger.getLogger(UserService.class);

    public void createUser() {
        logger.info("Creating user");
    }
}
```

```
}  
}
```

Good Example (Interface + Wrapper)

```
public interface AppLogger {  
    void info(String message);  
    void error(String message);  
}  
  
import org.apache.log4j.Logger;  
  
public class Log4jLogger implements AppLogger {  
    private final Logger logger;  
  
    public Log4jLogger(Class<?> clazz) {  
        this.logger = Logger.getLogger(clazz);  
    }  
  
    public void info(String message) {  
        logger.info(message);  
    }  
  
    public void error(String message) {  
        logger.error(message);  
    }  
}  
  
public class UserService {  
    private final AppLogger logger;  
  
    public UserService() {  
        this.logger = new Log4jLogger(UserService.class);  
    }  
  
    public void createUser() {  
        logger.info("Creating user");  
    }  
}
```

Benefits:

- Decoupled from logging library
- Easier to test
- Easy to replace logging framework

5. Using Code That Does Not Yet Exist

```
public interface Transmitter {  
    void transmit(double frequency, DataStream data);  
}  
  
public class TransmitterAdapter implements Transmitter {  
    private RealApi api;  
  
    public void transmit(double freq, DataStream data) {  
        api.send(freq, data);  
    }  
}
```

6. Clean Boundaries Principles

- Do not expose third-party APIs
- Encapsulate dependencies
- Use adapters when needed
- Write boundary tests
- Depend on what you control